

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ THI VÀ BÀI LÀM

Tên học phần: Toán ứng dụng CNTT

Mã học phần: Hình thức thi: *Tự luận có giám sát*

Đề số: **D0004** Thời gian làm bài: 90 phút (*không kể thời gian chép/phát đề*)

Được sử dụng tài liệu khi làm bài.

Họ tên:Hà Đức Kiên**Lớp:**.....23T_DT1.....**MSSV:**...102230194.....

Sinh viên làm bài trực tiếp trên tệp này, lưu tệp với định dạng MSSV_HọTên.pdf và nộp bài thông qua MS Teams.

Câu 1 (2.0 điểm): Viết chương trình (có sử dụng hàm) thực hiện công việc sau:

a) (1.0 điểm) Tìm số M là ước số nguyên tố lớn nhất của N, biết rằng $N=95777$.

Trả lời: Dán code bên dưới:

```
#include <iostream>
#include <math.h>
#include <vector>
#include <cstring>
#include <string>

using namespace std;

const float e = 2.718281828;

vector<int> primes, alphas;

bool check_primes(int n) {
    if(n <= 1) return false;
    for(int i = 2; i <= sqrt(n); i++) {
        if(n % i == 0) return false;
```

```

}

return true;

}

void create_primes(int n) {
for(int i = 1; i <= n; i++) {
if(check_primes(i)) primes.push_back(i);
}
}

void create_alphas(int n) {
for(int i = 0; i < primes.size() ; i++) {
int exponent = 0;
while(n != 1) {
if(n % primes[i] == 0) {
exponent++;
n /= primes[i];
} else break;
}
alphas.push_back(exponent);
}
}

int GCD_max(vector<int> primes, int n) {
for(int i = primes.size() - 1; i >= 0; i--) {
if(n % primes[i] == 0) {
return primes[i];
}
}
return 1;
}

int main() {
int n = 95777;

create_primes(n);

```

```

create_alphas(n);

int M = GCD_max(primes, n);

cout << "Uoc so nguyen to lon nhat cua " << n << " la " << M << endl;

return 0;

}

```

Trả lời: Dán kết quả thực thi vào bên dưới:

```

[Running] cd "/media/haduckien/E/Studying/HK3/Mathmetics for CS (3)/BTC1/"
Uoc so nguyen to lon nhat cua 95777 la 8707

```

b) (1.0 điểm) Tìm số lượng các số hoàn hảo nhỏ hơn M, liệt kê và tính tổng của chúng.

Trả lời: Dán code vào bên dưới:

```

#include <iostream>

#include <math.h>

#include <vector>

#include <cstring>

#include <string>

using namespace std;

const float e = 2.718281828;

vector<int> primes, alphas;

bool check_primes(int n) {
    if(n <= 1) return false;
    for(int i = 2; i <= sqrt(n); i++) {
        if(n % i == 0) return false;
    }
    return true;
}

```

```
void create_primes(int n) {  
    for(int i = 1; i <= n; i++) {  
        if(check_primes(i)) primes.push_back(i);  
    }  
}
```

```
void create_alphas(int n) {  
    for(int i = 0; i < primes.size(); i++) {  
        int exponent = 0;  
        while(n != 1) {  
            if(n % primes[i] == 0) {  
                exponent++;  
                n /= primes[i];  
            } else break;  
        }  
        alphas.push_back(exponent);  
    }  
}
```

```
bool isPerfect(int n) {  
    int sum = 0;  
    for(int i = 1; i < n; i++) {  
        if(n % i == 0) {  
            sum += i;  
        }  
    }  
    if(n == sum) return true;  
    else return false;  
}
```

```
int numberPerfect(int n) {  
    int count = 0;  
    for(int i = 1; i <= n; i++) {
```

```

if(isPerfect(i)) {
    cout << i << " ";
    count++;
}
}

return count;
}

int GCD_max(vector<int> primes, int n) {
    for(int i = primes.size() - 1; i >= 0; i--) {
        if(n % primes[i] == 0) {
            return primes[i];
        }
    }
    return 1;
}

int main() {
    int n = 95777;
    create_primes(n);
    create_alphas(n);
    int M = GCD_max(primes, n);
    cout << "Uoc so nguyen to lon nhat cua " << n << " la " << M << endl;
    cout << "Cac so hoan hao nho hon " << M << ":";
    int count = numberPerfect(M);
    cout << "\nSo hoan hao nho hon " << M << " co " << count << " so.";
    return 0;
}

```

Trả lời: Dán kết quả thực thi vào bên dưới:

```

[Running] cd "/media/haduckien/E/Studying/HK3/Mathmetics for CS (3)/BTC1/"
Uoc so nguyen to lon nhat cua 95777 la 8707
Cac so hoan hao nho hon 8707:6 28 496 8128
So hoan hao nho hon 8707 co 4 so.

```

Câu 2 (2.0 điểm): Cho ma trận A. Viết chương trình (có sử dụng hàm) thực hiện phân rã ma trận A bằng phương pháp SVD.

Trả lời: Dán code vào bên dưới

```
#include <iostream>
#include <Eigen/Dense>
#include <iomanip>
#include <cmath>

using namespace std;
using namespace Eigen;

void input(double A[][10], int row, int col);
void swap(double &a, double &b);
void display(double A[][10], int row, int col);
void chuyenvi(double A[][10], double At[][10], int rows, int cols);
void NhanMaTran(MatrixXd &S, double A[][10], double B[][10], int row1, int col1, int col2);
void GetEigenValuesAndVector(MatrixXd S, MatrixXd &lambda, MatrixXd &vector);
void tinhMatranS(MatrixXd lambda, double sigma[][10], int rows, int cols);
void tinhMatranU(MatrixXd lambda, MatrixXd vector, double U[][10], double A[][10], int rows, int cols);
void tinhMatranV(MatrixXd vector, double V[][10]);
void gramSchmidt(double U[][10], int rows, int cols);

int main() {
    int rows, cols;
    double A[10][10], At[10][10];
    cout << "ENTER MATRIX " << endl;
    cout << "ROWS: "; cin >> rows;
    cout << "COLUMNS: "; cin >> cols;
    cout << "ENTER MATRIX ELEMENTS: " << endl;
    input(A, rows, cols);
    cout << "MATRIX A" << endl;
    display(A, rows, cols);
    MatrixXd S(cols, cols), lambda(cols, 1), vector(cols, cols);
```

```

chuyenvi(A, At, rows, cols);

NhanMaTran(S, At, A, cols, rows, cols);

GetEigenValuesAndVector(S, lambda, vector);

double sigma[10][10], U[10][10], V[10][10];

tinhMatranU(lambda, vector, U, A, rows, cols);

cout << "MATRIX U" << endl;

display(U, rows, rows);

tinhMatranS(lambda, sigma, rows, cols);

cout << "MATRIX SIGMA" << endl;

display(sigma, rows, cols);

tinhMatranV(vector, V);

cout << "MATRIX VT" << endl;

double VT[10][10];

chuyenvi(V, VT, cols, cols);

display(VT, cols, cols);

return 0;

}

void input(double A[][10], int row, int col) {
for (int i = 0; i < row; i++)
for (int j = 0; j < col; j++){
cout << "a[" << i + 1 << "][" << j + 1 << "] = ";
cin >> A[i][j];
}
}

void display(double A[][10], int row, int col) {
for (int i = 0; i < row; i++)
{
for (int j = 0; j < col; j++)
cout << setw(9) << fixed << setprecision(4) << A[i][j];
cout << endl;
}
}

```

```
}
```

```
void swap(double &a, double &b) {
```

```
double temp = a;
```

```
a = b;
```

```
b = temp;
```

```
}
```

```
void chuyenvi(double A[][10], double At[][10], int rows, int cols) {
```

```
for (int i=0; i<rows; i++)
```

```
for (int j=0; j<cols; j++) {
```

```
At[j][i] = A[i][j];
```

```
}
```

```
}
```

```
void NhanMaTran(MatrixXd &S, double A[][10], double B[][10], int row1, int col1, int col2) {
```

```
for (int i=0; i<row1; i++) {
```

```
for (int j=0; j<col2; j++) {
```

```
S(i,j) = 0;
```

```
for (int k=0; k<col1; k++) {
```

```
S(i,j) = S(i,j) + A[i][k] * B[k][j];
```

```
}
```

```
}
```

```
}
```

```
}
```

```
void GetEigenValuesAndVector(MatrixXd S, MatrixXd &lambda, MatrixXd &vector) {
```

```
SelfAdjointEigenSolver<Eigen::MatrixXd> eigensolver(S);
```

```
vector = eigensolver.eigenvectors();
```

```
lambda = eigensolver.eigenvalues();
```

```
int k = lambda.rows();
```

```
int l = vector.rows();
```

```
for (int i=0; i<k; i++)
```



```

{
    if (lambda(i, 0) < 0.00001)
        lambda(i, 0) = 0;
}

for (int i = 0; i < k; i++) {
    for (int j = i + 1; j < k; j++)
    {
        if (lambda(j, 0) > lambda(i, 0))
        {
            swap(lambda(j, 0), lambda(i, 0));
            for (int h = 0; h < l; h++)
                swap(vector(h, i), vector(h, j));
        }
    }
}

void tinhMatranU(MatrixXd lambda, MatrixXd vector, double U[][10], double A[][10], int rows, int cols) {
    MatrixXd ui(rows, 1);
    double Vi[cols][10];
    for (int i = 0; i < cols; i++) {
        for (int j = 0; j < cols; j++) {
            Vi[j][0] = vector(j, i);
        }
        NhanMaTran(ui, A, Vi, rows, cols, 1);
        for (int k = 0; k < rows; k++) {
            if (lambda(i, 0) != 0) {
                U[k][i] = (1 / sqrt(lambda(i, 0))) * ui(k, 0);
            } else {
                U[k][i] = 0;
            }
        }
    }
}

```

```

if(rows > cols) {
if (rows > cols) {
MatrixXd U_matrix(rows, cols);
for (int i = 0; i < rows; i++) {
for (int j = 0; j < cols; j++) {
U_matrix(i, j) = U[i][j];
}
}

MatrixXd orthogonal_basis = U_matrix.householderQr().householderQ();
for (int j = cols; j < rows; j++) {
for (int i = 0; i < rows; i++) {
U[i][j] = orthogonal_basis(i, j);
}
}
}
} else {
gramSchmidt(U, rows, cols);
}

}

void gramSchmidt(double U[][10], int rows, int cols) {
for (int i = 0; i < cols; i++) {
for (int j = 0; j < i; j++) {
double dot_product = 0;
for (int k = 0; k < rows; k++) {
dot_product += U[k][i] * U[k][j];
}
for (int k = 0; k < rows; k++) {
U[k][i] -= dot_product * U[k][j];
}
}
}

double norm = 0;

```

```

for (int k = 0; k < rows; k++) {
    norm += U[k][i] * U[k][i];
}
norm = sqrt(norm);
if (norm > 1e-10) {
    for (int k = 0; k < rows; k++) {
        U[k][i] /= norm;
    }
} else {
    MatrixXd random_vector = MatrixXd::Random(rows, 1);
    for (int j = 0; j < i; j++) {
        double dot_product = 0;
        for (int k = 0; k < rows; k++) {
            dot_product += random_vector(k, 0) * U[k][j];
        }
        random_vector(k, 0) -= dot_product * U[k][j];
    }
}
double random_norm = random_vector.norm();
for (int k = 0; k < rows; k++) {
    U[k][i] = random_vector(k, 0) / random_norm;
}
}
}
}

void tinhMatranV(MatrixXd vector, double V[][10]) {
    int row = vector.rows(), col = vector.cols();
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
        {

```

```

V[i][j] = vector(i, j);
}
}
}

void tinhMatranS(MatrixXd lambda, double sigma[][10], int rows, int cols){
    int k = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
        {
            sigma[i][j] = (i != j) ? 0 : sqrt(lambda(k, 0));
            if (i == j)
                k++;
        }
}

```

$$A = \begin{bmatrix} 8 & 3 & 7 \\ 8 & 3 & 5 \\ 8 & 3 & 3 \end{bmatrix}, \text{ sai số } \varepsilon = 10^{-5}.$$

Trả lời: Dán kết quả thực thi vào bên dưới biết rằng

```

(.venv) haduckien@haduckien-Vivobook-ASUSLaptop-K3504VA-S3504VA: /media/haduckien/E/Studying/HK3/Mathmetics for C
ENTER MATRIX
ROWS: 3
COLUMNS: 3
ENTER MATRIX ELEMENTS:
a[1][1] = 8
a[1][2] = 3
a[1][3] = 7
a[2][1] = 8
a[2][2] = 3
a[2][3] = 5
a[3][1] = 8
a[3][2] = 3
a[3][3] = 3
MATRIX A
  8.0000  3.0000  7.0000
  8.0000  3.0000  5.0000
  8.0000  3.0000  3.0000
MATRIX U
  0.6352  0.6557  0.4082
  0.5753 -0.0489 -0.8165
  0.5154 -0.7535  0.4082
MATRIX SIGMA
 17.2071  0.0000  0.0000
  0.0000  2.4325  0.0000
  0.0000  0.0000  0.0000
MATRIX VT
  0.8024  0.3009  0.5154
 -0.4826 -0.1810  0.8569
 -0.3511  0.9363  0.0000

```

Câu 3 (3.0 điểm): Cho mười lăm điểm trong không gian Oxy như sau: {3,5}, {6,8}, {9,10}, {11,2}, {4,9}, {7,8}, {9,9}, {10,3}, {11,15}, {34,5}, {32,1}, {25,10}, {18,8}, {7,18}, {10,14}.

a) (1.0 điểm) Mô tả thuật toán xác định bao lồi của tập điểm đã cho

Trả lời: dàn sơ đồ khối hoặc mã giả:

Sử dụng thuật toán Chain - độ phức tạp $O(n \log n)$

function chansalgorithm(points):

for t = 0 to infinity: // Tăng dần số lần thử nghiệm

max_hulls = $2^{(2^t)}$ // Số phân đoạn tối đa

for m = 1 to max_hulls: // Số điểm trong mỗi phân đoạn

hulls = [] // Danh sách các hull con

for i = 0 to size(points) step m: // Chia tập hợp thành các phân đoạn

if i + m <= size(points):

chunk = points[i to i + m] // Lấy phân đoạn m điểm

else:

chunk = points[i to end] // Lấy phần còn lại

hulls.append(GrahamScan(chunk)) // Tìm hull của phân đoạn và thêm vào hulls

hull = [] // Hull cuối cùng

hull.append(extreme_hullpt_pair(hulls)) // Tìm điểm cực trị ban đầu

for i = 0 to m: // Tìm tối đa m điểm trong hull

p = next_hullpt_pair(hulls, hull[last]) // Tìm điểm tiếp theo trên hull

if p == hull[0]: // Nếu điểm tiếp theo quay về điểm đầu

output = []

for j = 0 to size(hull):

output.append(hulls[hull[j].first][hull[j].second]) // Thêm điểm vào output

output.pop_back() // Loại bỏ điểm cuối (trùng điểm đầu)

return output // Trả về hull cuối cùng

hull.append(p) // Thêm điểm tiếp theo vào hull

return [] // Trả về rỗng nếu không tìm thấy

b) (1.0 điểm) Xác định bao lồi và cạnh nhỏ nhất của đa giác lồi vừa tìm được

Trả lời: Dán code bên dưới:

```
#include <iostream>

#include <stdlib.h>

#include <vector>

#include <algorithm>

#include <utility>

#include <set>

#include <cmath>

#include <limits>

#include <ctime>

#define RIGHT_TURN -1

#define LEFT_TURN 1

#define COLLINEAR 0

using namespace std;

class Point{

public:

int x;

int y;

Point (int newx=0,int newy=0){

x=newx;

y=newy;

}

friend bool operator== (const Point& p1,const Point& p2){

return (p1.x==p2.x && p1.y==p2.y);

}

friend bool operator!= (const Point& p1,const Point& p2){

return (!(p1.x==p2.x && p1.y==p2.y));

}

friend ostream& operator<<(ostream& output,const Point& p){

output<<"("<<p.x<<" "<<p.y<<")";

return output;

}
```

```
} p0;
```

```
int orientation(Point p, Point q, Point r){
```

```
int val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
```

```
if (val == 0) return 0;
```

```
return (val > 0)? -1: 1;
```

```
}
```

```
double dist(Point p1, Point p2){
```

```
return sqrt((p1.x - p2.x)*(p1.x - p2.x) + (p1.y - p2.y)*(p1.y - p2.y));
```

```
}
```

```
pair<Point, Point> find_min_edge(vector<Point> v){
```

```
double min_dist = numeric_limits<double>::max();
```

```
int n = v.size();
```

```
pair<Point, Point> res;
```

```
for (int i = 0; i < n; i++){
```

```
int j = (i + 1) % n;
```

```
double distance = dist(v[i], v[j]);
```

```
if(distance < min_dist){
```

```
min_dist = distance;
```

```
res = {v[i], v[j]};
```

```
}
```

```
}
```

```
return res;
```

```
}
```

```
pair<Point, Point> find_closet_points(vector<Point> v, int n){
```

```
double min_dist = numeric_limits<double>::max();
```

```
pair<Point, Point> closet_pair;
```

```
for (int i = 0; i < n; i++){
```

```
for(int j = i + 1; j < n; j++){
```

```
double distance = dist(v[i], v[j]);
```

```
if(distance < min_dist){
```

```
min_dist = distance;
```

```

    closet_pair = {v[i], v[j]};
}
}
}

return closet_pair;
}

int compare(const void *vp1, const void *vp2){
    Point *p1 = (Point *)vp1;
    Point *p2 = (Point *)vp2;

    int orient = orientation(p0, *p1, *p2);

    if (orient == 0)

        return (dist(p0, *p2) >= dist(p0, *p1)) ? -1 : 1;

    return (orient == 1) ? -1 : 1;
}

int tangent(vector<Point> v, Point p){
    int l=0;

    int r= v.size();

    int l_before = orientation(p, v[0], v[v.size()-1]);

    int l_after = orientation(p, v[0], v[(l + 1) % v.size()]);

    while (l < r){

        int c = ((l + r)>>1);

        int c_before = orientation(p, v[c], v[(c - 1) % v.size()]);

        int c_after = orientation(p, v[c], v[(c + 1) % v.size()]);

        int c_side = orientation(p, v[l], v[c]);

        if ((c_before != RIGHT_TURN) && (c_after != RIGHT_TURN))

            return c;

        else if ((c_side == LEFT_TURN) && (l_after == RIGHT_TURN || l_before == l_after) || (c_side == RIGHT_TURN
        && c_before == RIGHT_TURN))

            r = c;

        else

            l = c + 1;

        l_before = -c_after;

        l_after = orientation(p, v[l], v[(l + 1) % v.size()]);
    }
}

```



```

}

return l;

}

pair<int,int> extreme_hullpt_pair(vector<vector<Point> >& hulls){
int h= 0,p= 0;
for (int i=0; i<hulls.size(); ++i){
int min_index=0, min_y = hulls[i][0].y;
for(int j=1; j< hulls[i].size(); ++j){
if(hulls[i][j].y < min_y){
min_y=hulls[i][j].y;
min_index=j;
}
}
if(hulls[i][min_index].y < hulls[h][p].y){
h=i;
p=min_index;
}
}
return make_pair(h,p);
}

pair<int,int> next_hullpt_pair(vector<vector<Point> >& hulls, pair<int,int> lPoint){
Point p = hulls[lPoint.first][lPoint.second];
pair<int,int> next = make_pair(lPoint.first, (lPoint.second + 1) % hulls[lPoint.first].size());
for (int h=0; h< hulls.size(); h++){
if(h != lPoint.first){
int s= tangent(hulls[h],p);
Point q= hulls[next.first][next.second];
Point r= hulls[h][s];
int t= orientation(p,q,r);
if( t== RIGHT_TURN || (t==COLLINEAR) && dist(p,r)>dist(p,q))
next = make_pair(h,s);
}
}
}

```

```

return next;

}

vector<Point> keep_left (vector<Point>& v, Point p){
while(v.size()>1 && orientation(v[v.size()-2],v[v.size()-1],p) != LEFT_TURN)
v.pop_back();
if(!v.size() || v[v.size()-1] != p)
v.push_back(p);
return v;
}

vector<Point> GrahamScan(vector<Point>& Points) {
if (Points.size() <= 1)
return Points;

qsort(&Points[0], Points.size(), sizeof(Point), compare);

vector<Point> lower_hull;
for (const auto& p : Points)
lower_hull = keep_left(lower_hull, p);
reverse(Points.begin(), Points.end());

vector<Point> upper_hull;
for (const auto& p : Points)
upper_hull = keep_left(upper_hull, p);
lower_hull.insert(lower_hull.end(), upper_hull.begin() + 1, upper_hull.end());
return lower_hull;
}

vector<Point> chansalgorithm(vector<Point> v) {
for(int t = 0; t < v.size(); ++t) {
for(int m = 1; m < (1 << (1 << t)); ++m) {
vector<vector<Point>> hulls;
for(int i = 0; i < v.size(); i += m) {
vector<Point> chunk;
if(v.begin() + i + m <= v.end())
chunk.assign(v.begin() + i, v.begin() + i + m);
else
chunk.assign(v.begin() + i, v.end());

```

```

hulls.push_back(GrahamScan(chunk));
}

vector<pair<int, int>> hull;

hull.push_back(extreme_hullpt_pair(hulls));

for(int i = 0; i < m; ++i) {

pair<int, int> p = next_hullpt_pair(hulls, hull[hull.size() - 1]);

vector<Point> output;

if(p == hull[0]) {

for(int j = 0; j < hull.size(); ++j) {

output.push_back(hulls[hull[j].first][hull[j].second]);

}

output.pop_back();

return output;

}

hull.push_back(p);

}

}

}

return {};

}

nt main(){

vector<Point> v = {{3,5}, {6,8},{9,10},
{11,2},{4,9},{7,8},{9,9},{10,3},{11,15},{34,5},{32,1},{25,10},{18,8},{7,18},{10,14}};

cout << "Original Points: \n";

for (auto p : v) cout << p << "\t";

cout << endl;

vector<Point> output = chansalgorithm(v);

cout << "Convex hull: \n";

for(auto p : output) cout << p << "\t";

cout << "\n";

cout << "Polygon area: " << poly_area(output) << endl;

pair<Point, Point> min_edge = find_min_edge(output);

cout << "Shortest edge in Convex Hull: (" << min_edge.first.x << ", " << min_edge.first.y << ") and ("

```

```

<< min_edge.second.x << " , " << min_edge.second.y << ")\n";

cout << "Length of shortest edge: " << dist(min_edge.first, min_edge.second) << endl;

return 0;

}

```

Trả lời: Dán kết quả thực thi vào bên dưới:

```

[Running] cd "/media/haduckien/E/Studyng/HK3/Mathmetics for CS (3)/BTC7/" && g++ Chanalgorithm.cpp -o Chanalgorithm && "/media/haduckien/E/Studyng/HK3/Mathmetics for CS (3)/BTC7/" ./Chanalgorithm
Original Points:
(3,5) (6,8) (9,10) (11,2) (4,9) (7,8) (9,9) (10,3) (11,15) (34,5) (32,1) (25,10) (18,8) (7,18) (10,14)
Convex hull:
(32,1) (34,5) (25,10) (7,18) (4,9) (3,5) (11,2)
Polygon area: 301.5
Shortest edge in Convex Hull: (4, 9) and (3, 5)
Length of shortest edge: 4.12311

```

c) (1.0 điểm) Xác định số lượng các điểm nằm bên trong bao lồi và liệt kê chúng

Trả lời: Dán code bên dưới:

```

bool is_In_Vector(const vector<Point>& v, const Point& p) {

    int n = v.size();

    for (int i = 0; i < n; i++) {

        if (v[i] == p) {

            return true;

        }

    }

    return false;

}

vector<Point> elements_In_A_And_Not_In_B(const vector<Point>& A, const vector<Point>& B) {

    vector<Point> result;

    for (const auto& point : A) {

        if (!is_In_Vector(B, point)) {

            result.push_back(point);

        }

    }

    return result;

}

int main(){

    vector<Point> v = {{3,5}, {6,8}, {9,10},
    {11,2}, {4,9}, {7,8}, {9,9}, {10,3}, {11,15}, {34,5}, {32,1}, {25,10}, {18,8}, {7,18}, {10,14}};

    cout << "Original Points: \n";

```

```

for (auto p : v) cout << p << "\t";

cout << endl;

vector<Point> output = chansalgorithm(v);

cout << "Convex hull: \n";

for(auto p : output) cout << p << "\t";

cout << "\n";

cout << "Polygon area: " << poly_area(output) << endl;

pair<Point, Point> min_edge = find_min_edge(output);

cout << "Shortest edge in Convex Hull: (" << min_edge.first.x << ", " << min_edge.first.y << ") and ("
<< min_edge.second.x << ", " << min_edge.second.y << ") \n";

cout << "Length of shortest edge: " << dist(min_edge.first, min_edge.second) << endl;

cout << "Points inside Convex Hull: \n";

vector<Point> points_inside = elements_In_A_And_Not_In_B(v, output);

for (auto p : points_inside) cout << p << "\t";

cout << endl;

return 0;

}

```

Trả lời: Dán kết quả thực thi vào bên dưới:

```

[Running] cd "/media/haduckien/E/Studyng/HK3/Mathmetics for CS (3)/BTC7/" && g++ Chanalgorithm.cpp -o Chanalgorithm && "/media/haduck:
Original Points:
(3,5) (6,8) (9,10) (11,2) (4,9) (7,8) (9,9) (10,3) (11,15) (34,5) (32,1) (25,10) (18,8) (7,18) (10,14)
Convex hull:
(32,1) (34,5) (25,10) (7,18) (4,9) (3,5) (11,2)
Polygon area: 301.5
Shortest edge in Convex Hull: (4, 9) and (3, 5)
Length of shortest edge: 4.12311
Points inside Convex Hull:
(6,8) (9,10) (7,8) (9,9) (10,3) (11,15) (18,8) (10,14)

```

Câu 4 (2.0 điểm): Cho hàm số
$$f(x) = \frac{\log_{10}(2x^2 + 1) + e^{8x} + 14x^2 - 6x}{2x^2 + 1} - 4x^2$$

- a) (1.0 điểm) Trình bày thuật toán tối ưu hàm số đã cho sử dụng phương pháp *gradient descent* với *momentum*, biết rằng tham số học (learning rate) γ , hệ số động lượng là α .

Trả lời: dán sơ đồ khối hoặc mã giả:

```

function optimize(gamma, alpha, x_init, objFunc, gradFunc):

```

```

    x = x_init                // Khởi tạo giá trị x ban đầu

```

```

    v = 0.0                  // Vận tốc ban đầu

```

```

    iterations = 0           // Đếm số vòng lặp

```

```

while iterations < 1000:      // Lặp tối đa 1000 lần

    iterations = iterations + 1    // Tăng số vòng lặp

    x_prev = x                // Lưu giá trị x trước đó

    v = alpha * v + (1 - alpha) * gradFunc(x) // Cập nhật vận tốc

    x = x - gamma * v          // Cập nhật giá trị x

    print("Iteration ", iterations,

          ": x = ", x,

          ", Objective value = ", objFunc(x))

    if abs(x - x_prev) < 1e-5:    // Kiểm tra hội tụ

        print("Optimization completed.")

        print("Optimal x: ", x)

        print("Optimal objective value: ", objFunc(x))

        break                    // Thoát vòng lặp nếu hội tụ

if iterations == 1000:          // Kiểm tra số lần lặp tối đa

    print("Reached maximum iterations without convergence.")

```

- b) (1.0 điểm) Viết chương trình (có dùng hàm) tính giá trị bé nhất của $f(x)$ sử dụng phương pháp *gradient descent* với *momentum* với số bước lặp N và sai số ε .

Trả lời: Dán code vào bên dưới:

```

#include <iostream>

#include <cmath>

using namespace std;

double function1(double x) {

    return (log10(2*x*x + exp(8*x)+14*x*x - 6*x))/(2*x*x+1) - 4*x*x;

}

double derivativeFunction1(double x) {

    return ((4*x*(log10(2*x*x+1)+exp(8*x)+14*x*x-6*x))-

    (2*x*x+1)*(((4*x)/(log(10)*(2*x*x+1)))+8*exp(8*x)+28*x-6))/((2*x*x+1)*(2*x*x+1)) - 8*x;

}

```

```

void optimize(double gamma, double alpha, double x_init, double (*objFunc)(double), double (*gradFunc)(double)) {
    double x = x_init;
    double v = 0.0;
    int iterations = 0;
    while (iterations < 1000) {
        iterations++;
        double x_prev = x;
        v = alpha * v + (1 - alpha) * gradFunc(x);
        x = x - gamma * v;

        cout << "Objective function value at iteration " << iterations << ": x = " << x << ", function value = " <<
        objFunc(x) << endl;

        if (fabs(x - x_prev) < 1e-5) {
            cout << "Optimization of the objective function completed." << endl;
            cout << "Optimal value of x: " << objFunc(x) << endl;
            break;
        }
    }

    if (iterations == 1000) {
        cout << "Reached maximum iterations without convergence." << endl;
    }
}

int main() {
    double gamma = 0.001;
    double alpha = 0.01;
    double x_init = -2.0;
    optimize(gamma, alpha, x_init, function1, derivativeFunction1);
    return 0;
}

```

Trả lời: Dán kết quả thực thi với điểm khởi $x = -2$, tham số học học (*learning rate*)

$\gamma = 0.001$, hệ số động lượng (*momentum coefficient*) là $\alpha = 0.01$, số bước lặp $N \geq 1000$ và sai số $\varepsilon = 10^{-5}$:

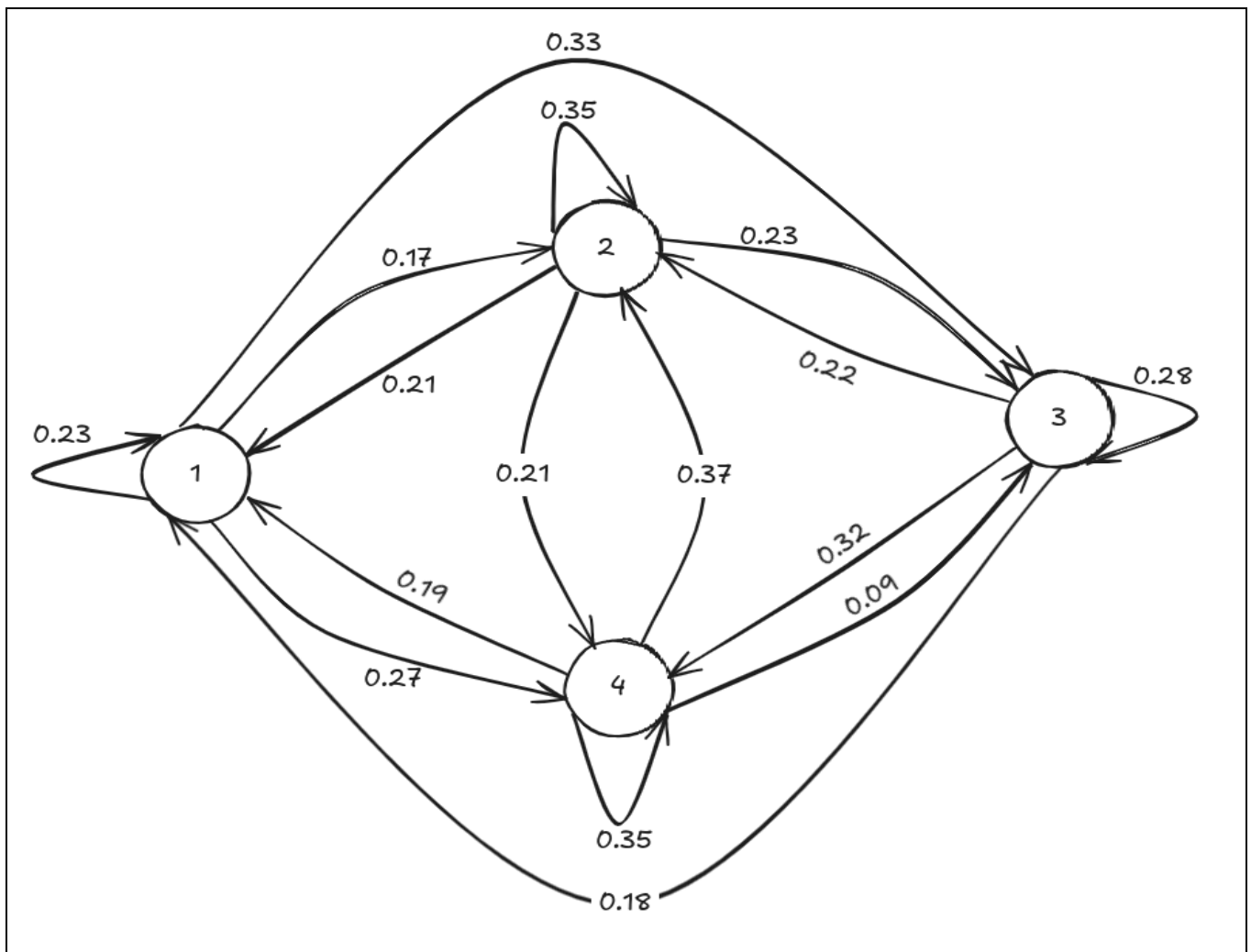
```
Objective function value at iteration 997: x = -5622.96, function value = -1.28471e+08  
Objective function value at iteration 998: x = -5667.96, function value = -1.28503e+08  
Objective function value at iteration 999: x = -5713.3, function value = -1.30567e+08  
Objective function value at iteration 1000: x = -5759, function value = -1.32664e+08  
Reached maximum iterations without convergence.
```

Câu 5 (1.0 điểm): Một hệ thống có chế độ làm việc ở mỗi giai đoạn vận hành chỉ với các trạng thái từ 1 đến 4. Chế độ làm việc của hệ thống này được mô tả bằng ma trận chuyển như sau:

$$P = \begin{bmatrix} 0.23 & 0.17 & 0.33 & 0.27 \\ 0.21 & 0.35 & 0.23 & 0.21 \\ 0.18 & 0.22 & 0.28 & 0.32 \\ 0.19 & 0.37 & 0.09 & 0.35 \end{bmatrix}$$

a) (0.5 điểm) Vẽ đồ thị biểu diễn chuỗi Markov tương ứng đã cho

Trả lời: Dán kết quả vào bên dưới



b) (0.5 điểm) Giả sử rằng hệ thống bắt đầu học ở trạng thái 2. Tính xác suất hệ thống làm việc ở trạng thái 4 sau hai và bốn bước thời gian vận hành.

Trả lời: Dán kết quả tính toán vào bên dưới:

Ma trận chuyển đổi trạng thái:

0.23	0.17	0.33	0.27
0.21	0.35	0.23	0.21
0.18	0.22	0.28	0.32
0.19	0.37	0.09	0.35

Ta có công thức tổng quát:

$$\pi(n+1) = \pi(n) * P = \pi(0) * P^{n+1}$$

Ta có ma trận chuyển đổi trạng thái P sau 3 giai đoạn vận hành là:

$$P^2 = \begin{bmatrix} 0.23 & 0.17 & 0.33 & 0.27 \\ 0.21 & 0.35 & 0.23 & 0.21 \\ 0.18 & 0.22 & 0.28 & 0.32 \\ 0.19 & 0.37 & 0.09 & 0.35 \end{bmatrix} * \begin{bmatrix} 0.23 & 0.17 & 0.33 & 0.27 \\ 0.21 & 0.35 & 0.23 & 0.21 \\ 0.18 & 0.22 & 0.28 & 0.32 \\ 0.19 & 0.37 & 0.09 & 0.35 \end{bmatrix}$$

$$P^2 = \begin{bmatrix} 0.1993 & 0.2711 & 0.2317 & 0.2979 \\ 0.2031 & 0.2865 & 0.2331 & 0.2773 \\ 0.1988 & 0.2876 & 0.2172 & 0.2964 \\ 0.2041 & 0.3111 & 0.2045 & 0.2803 \end{bmatrix}$$

Ta có vector phân phối xác suất tại thời điểm bắt đầu làm việc là:

$$\pi(0) = [0 \quad 1 \quad 0 \quad 0] \text{ (vì hệ thống bắt đầu ở trạng thái 2)}$$

Vậy vector phân phối sau giai đoạn vận hành là:

$$\pi(2) = \pi(0) * P^2 = [0 \quad 1 \quad 0 \quad 0] * \begin{bmatrix} 0.1993 & 0.2711 & 0.2317 & 0.2979 \\ 0.2031 & 0.2865 & 0.2331 & 0.2773 \\ 0.1988 & 0.2876 & 0.2172 & 0.2964 \\ 0.2041 & 0.3111 & 0.2045 & 0.2803 \end{bmatrix}$$

$$\pi(2) = [0.2301 \quad 0.2865 \quad 0.2331 \quad 0.2773]$$

Vậy xác suất hệ thống làm việc ở trạng thái 4 sau hai bước thời gian vận hành là: 0.2773

Ta có ma trận chuyển đổi trạng thái P sau 4 giai đoạn vận hành là:

$$P^4 = \begin{bmatrix} 0.23 & 0.17 & 0.33 & 0.27 \\ 0.21 & 0.35 & 0.23 & 0.21 \\ 0.18 & 0.22 & 0.28 & 0.32 \\ 0.19 & 0.37 & 0.09 & 0.35 \end{bmatrix} * \begin{bmatrix} 0.23 & 0.17 & 0.33 & 0.27 \\ 0.21 & 0.35 & 0.23 & 0.21 \\ 0.18 & 0.22 & 0.28 & 0.32 \\ 0.19 & 0.37 & 0.09 & 0.35 \end{bmatrix} *$$

$$\begin{bmatrix} 0.23 & 0.17 & 0.33 & 0.27 \\ 0.21 & 0.35 & 0.23 & 0.21 \\ 0.18 & 0.22 & 0.28 & 0.32 \\ 0.19 & 0.37 & 0.09 & 0.35 \end{bmatrix} * \begin{bmatrix} 0.23 & 0.17 & 0.33 & 0.27 \\ 0.21 & 0.35 & 0.23 & 0.21 \\ 0.18 & 0.22 & 0.28 & 0.32 \\ 0.19 & 0.37 & 0.09 & 0.35 \end{bmatrix}$$

$$P^4 = \begin{bmatrix} 0.2016 & 0.2910 & 0.2206 & 0.2867 \\ 0.2016 & 0.2904 & 0.2212 & 0.2868 \\ 0.2017 & 0.2909 & 0.2208 & 0.2864 \\ 0.2017 & 0.2904 & 0.2215 & 0.2862 \end{bmatrix}$$

Ta có vector phân phối xác suất tại thời điểm bắt đầu làm việc là:

$$\pi(0) = [0 \quad 1 \quad 0 \quad 0] \text{ (vì hệ thống bắt đầu ở trạng thái 2)}$$

Vậy vector phân phối sau giai đoạn vận hành là:

$$\pi(4) = \pi(0) * P^4 = \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0.2016 & 0.2910 & 0.2206 & 0.2867 \\ 0.2016 & 0.2904 & 0.2212 & 0.2868 \\ 0.2017 & 0.2909 & 0.2208 & 0.2864 \\ 0.2017 & 0.2904 & 0.2215 & 0.2862 \end{bmatrix}$$

$$\pi(4) = [0.2016 \quad 0.2904 \quad 0.2212 \quad 0.2868]$$

Vậy xác suất hệ thống làm việc ở trạng thái 4 sau bốn bước thời gian vận hành là: 0.2867

GIẢNG VIÊN BIÊN SOẠN ĐỀ THI

Đà Nẵng, ngày 4 tháng 12 năm 2024
KHOA CÔNG NGHỆ THÔNG TIN

(đã duyệt)